

9. DFT and FFT Algorithms

Experiment 9.1

// Test Case: Discrete Fourier Transform (DFT) of a Discrete-Time Signal

// Define the discrete time signal

x_n = [1, 2, 3, 4]; // Signal x[n]

// Compute the Discrete Fourier Transform (DFT)

X_k = fft(x_n); // Compute the DFT using Scilab's fft function

// Define the frequency axis

N = length(x_n); // Number of points in the DFT

k = 0:N-1; // Frequency index

// Initialize arrays to store magnitude and phase

magnitude = abs(X_k); // Compute magnitude of DFT

phase = zeros(1, N); // Initialize phase array

// Compute phase manually using atan

for i = 1:N

real_part = real(X_k(i));

imag_part = imag(X_k(i));

if real_part > 0 then

phase(i) = atan(imag_part / real_part); // Phase in $[-\pi/2, \pi/2]$

elseif real_part < 0 then

phase(i) = atan(imag_part / real_part) + %pi; // Adjust phase for left half-plane

else

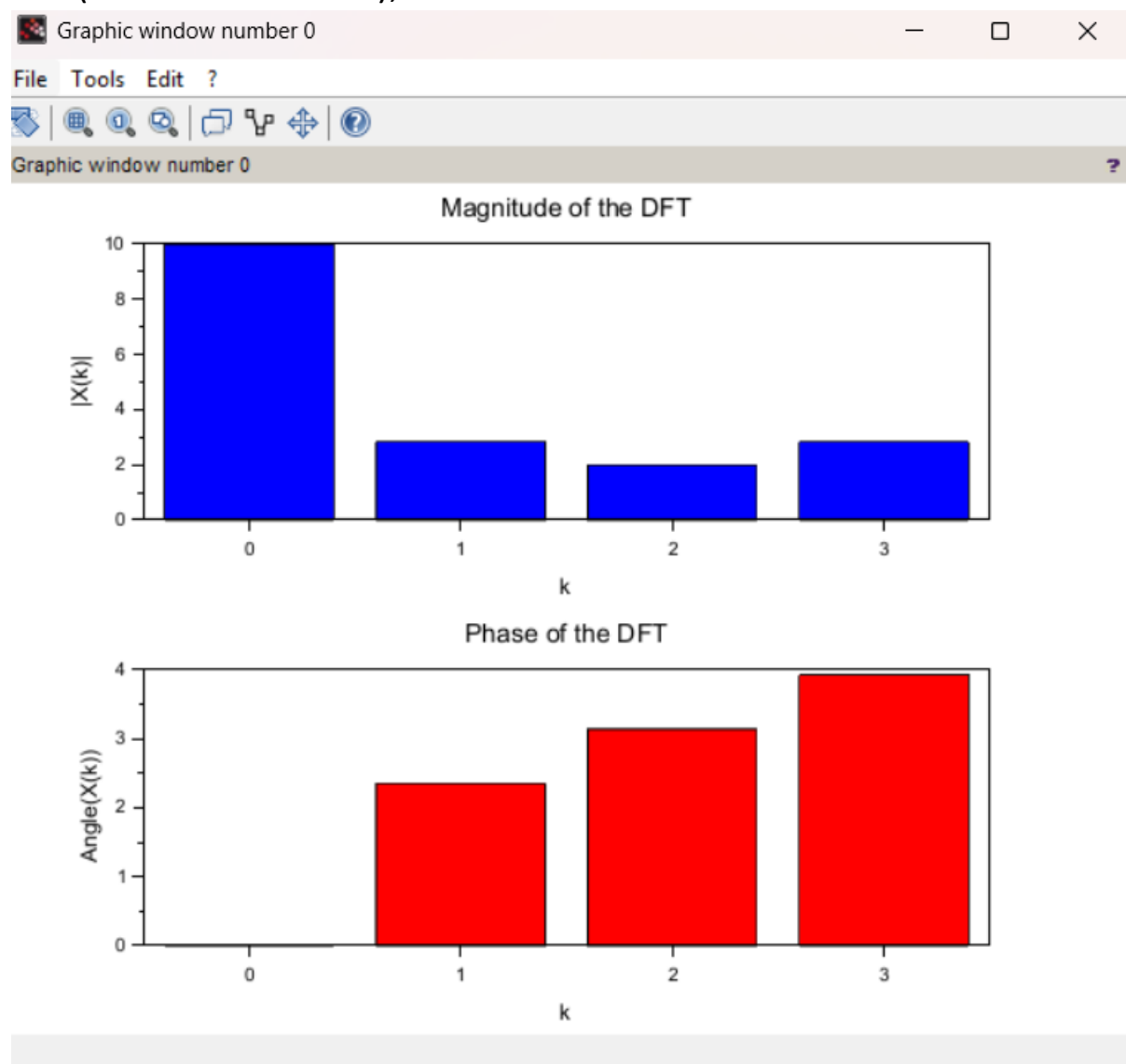
```
if imag_part > 0 then
    phase(i) = %pi/2; // Positive imaginary axis
elseif imag_part < 0 then
    phase(i) = -%pi/2; // Negative imaginary axis
else
    phase(i) = 0; // Origin case
end
end
end

// Plot the magnitude and phase of the DFT
clf; // Clear the current figure
// Plot magnitude
subplot(2,1,1);
bar(k, magnitude, 'b');
xlabel('k');
ylabel('|X(k)|');
title('Magnitude of the DFT');

// Plot phase
subplot(2,1,2);
bar(k, phase, 'r');
xlabel('k');

ylabel('Angle(X(k))');
```

```
title('Phase of the DFT');
```



```
clc;
```

```
clear;
```

```
clf;
```

```
//=====
```

```
// Circular Convolution of Discrete-Time Signals
```

```
//=====
```

```

// Input Signals
x_n = [1 2 3 4];
h_n = [2 1];

// Length of Circular Convolution
N = length(x_n);

// Zero-pad h[n]
h_n_padded = [h_n zeros(1, N-length(h_n))];

//-----
// Circular Convolution
//-----

y_n = zeros(1, N);

for n = 1:N
    sum_val = 0;
    for k = 1:N
        index = modulo(n-k, N) + 1;
        sum_val = sum_val + x_n(k) * h_n_padded(index);
    end
    y_n(n) = sum_val;
end
end

```

```
//-----
```

```
// Time Index
```

```
//-----
```

```
n = 0:N-1;
```

```
//-----
```

```
// Plot x[n]
```

```
//-----
```

```
subplot(3,1,1);
```

```
plot2d3(n, x_n);
```

```
xlabel("n");
```

```
ylabel("x[n]");
```

```
title("Signal x[n]");
```

```
//-----
```

```
// Plot h[n]
```

```
//-----
```

```
subplot(3,1,2);
```

```
plot2d3(n, h_n_padded);
```

```
xlabel("n");
```

```
ylabel("h[n]");
```

```
title("Signal h[n] (Padded)");
```

```
//-----
```

```
// Plot Circular Convolution
```

```
//-----
```

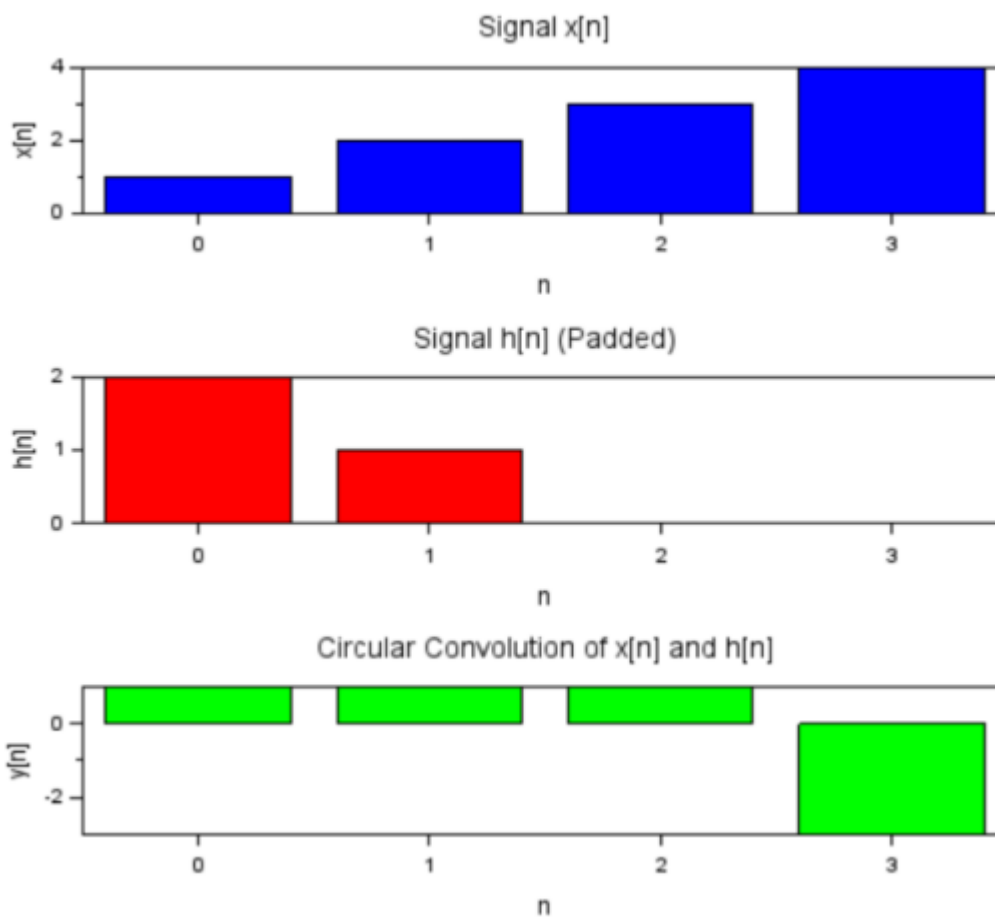
```
subplot(3,1,3);
```

```
plot2d3(n, y_n);
```

```
xlabel("n");
```

```
ylabel("y[n]");
```

```
title("Circular Convolution of x[n] and h[n]");
```

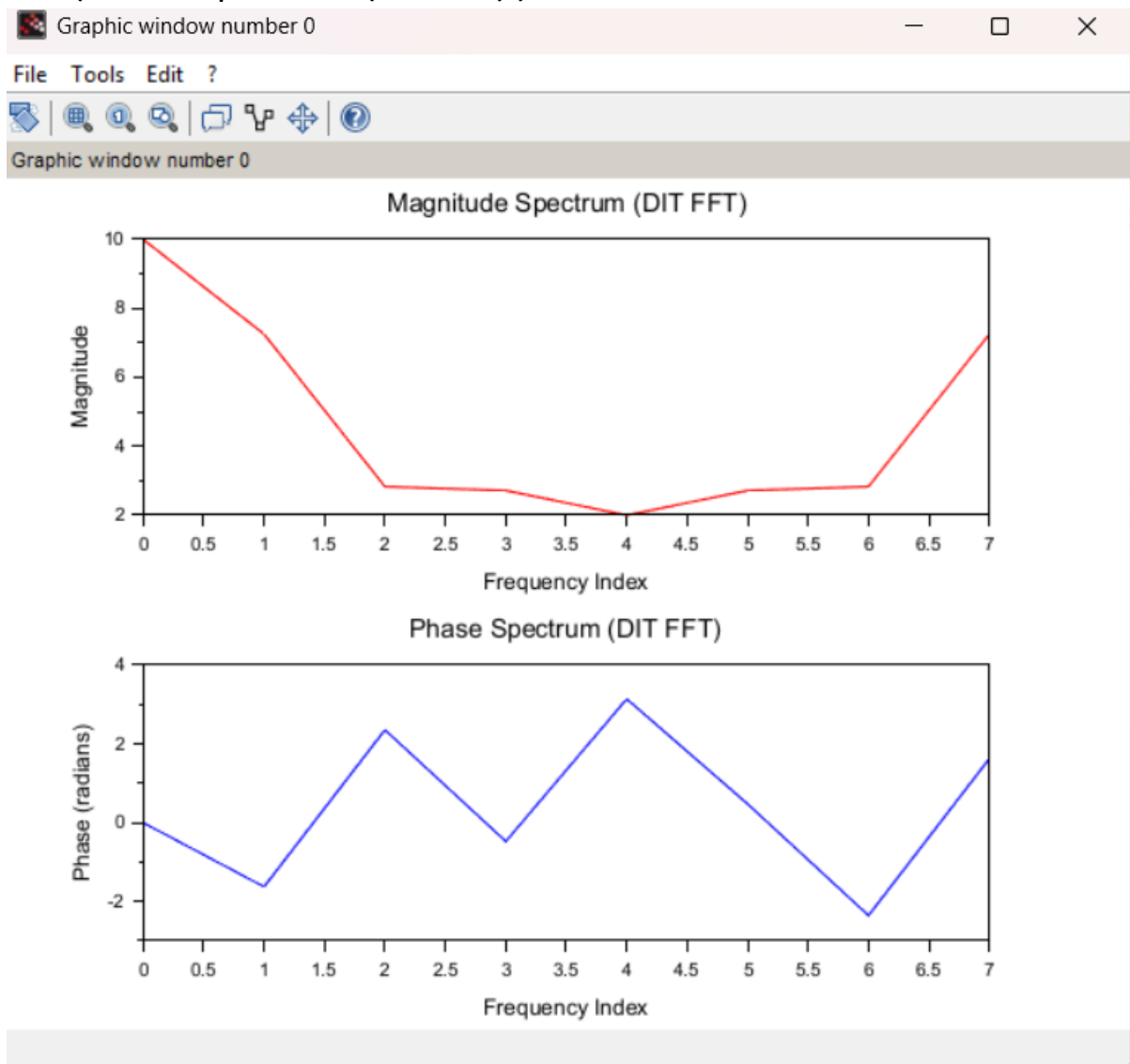


Experiment 9.3

```
clc;
clear;

// Input sequence (length should be power of 2)
x = [1, 2, 3, 4, 0, 0, 0, 0];
N = length(x);
X = fft(x, -1); // Scilab built-in FFT (DIT equivalent)
// Display results
disp("FFT (DIT) Result:");
disp(X);
subplot(2,1,1);
plot(0:N-1, abs(X), 'r');
xlabel('Frequency Index');
ylabel('Magnitude');
title('Magnitude Spectrum (DIT FFT)');
subplot(2,1,2);
phaseX = atan(imag(X), real(X)); // Phase in radians
plot(0:N-1, phaseX, 'b');
xlabel('Frequency Index');
ylabel('Phase (radians)');
```

```
title('Phase Spectrum (DIT FFT)');
```



Experiment 9.4

```
clc;
```

```
clear;
```

```
x = [1, 2, 3, 4, 0, 0, 0, 0];
```

```
N = length(x);
```

```
X = fft(x, -1); // DIF output is same as FFT in result
```

```
disp("FFT (DIF) Result:");
```

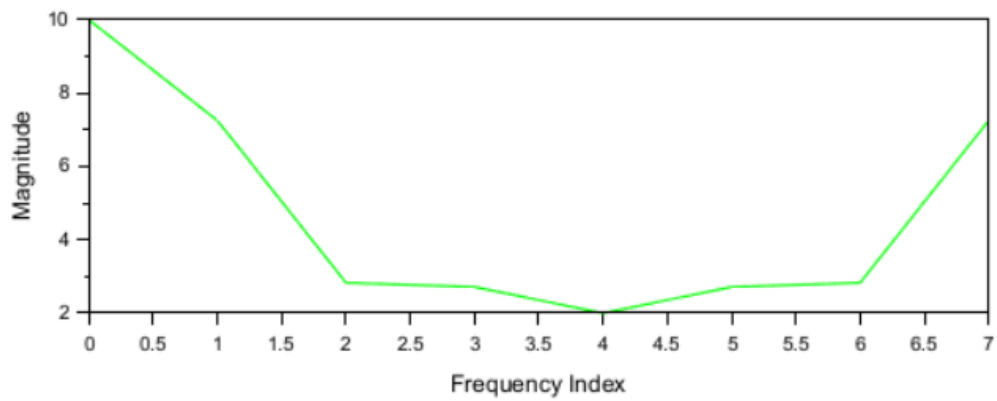
```
disp(X);
```



```
subplot(2,1,1);  
plot(0:N-1, abs(X), 'g');  
xlabel('Frequency Index');  
ylabel('Magnitude');  
title('Magnitude Spectrum (DIF FFT)');  
subplot(2,1,2);  
phaseX = atan(imag(X), real(X)); // Phase in radians  
plot(0:N-1, phaseX, 'm');  
xlabel('Frequency Index');  
ylabel('Phase (radians)');  
title('Phase Spectrum (DIF FFT)');
```



Magnitude Spectrum (DIF FFT)



Phase Spectrum (DIF FFT)

